Mastering Java

A Compressed Book for Experienced Programmers

Patrick Lin

July 22, 2026

Contents

Introduction: What the Compiler Proves and the Running Program Does

Java source participates in several distinct mechanisms. The language compiler checks declarations, expression types, accessibility, definite assignment, and other source rules. The virtual machine loads, links, initializes, and executes classes. Objects preserve identity while reference values move through variables, fields, arrays, and parameters. Library contracts add meanings—such as collection equality, resource ownership, or executor shutdown—that the language alone does not invent. Concurrent threads add ordering and visibility constraints that a locally correct method cannot settle by itself. Reliable Java reasoning begins by naming which mechanism performs an operation, on which value or object, during which phase, and what failure remains possible elsewhere.

This book is compressed by design. Each chapter states a durable account, then makes the account retrievable through committed predictions, exact evidence, causal explanations, an independent lab, delayed questions, and a short oral presentation. Do not read a prediction case as ordinary exposition. Stop before its result, write the result you expect, and name the rule responsible. A wrong prediction is useful evidence only if it remains visible long enough to locate the first mistaken step.

The route has five ordered regions. **Values and execution (Chapters 1–4)** establishes storage, identity, expressions, control flow, and calls. **Object and type models (Chapters 5–8)** connects construction, inheritance, domain forms, and erased generics. **Data, effects, and failure (Chapters 9–12)** applies those mechanisms to equality, collections, resources, exceptions, lambdas, and streams. **From source to concurrent run time (Chapters 13–16)** follows code through class loading, modules, the memory model, and task execution. **Production correctness (Chapters 17–20)** tests the combined model at external boundaries and in a concurrent capstone. The detailed contents are not a list of independent features; each region changes what the conclusions of the preceding regions mean in a larger program.

Pedagogical diagnosis and design decision

The intended reader can often make Java code compile but cannot always say why a nearby variation fails, which overload is selected, whether two paths reach the same object, when a generic guarantee disappears, or what another thread may observe. Syntax-first tutorials help a new programmer produce the first program. Encyclopedic references help a reader find a named feature. Recipe collections help reproduce a familiar solution. None directly repairs a model assembled from slogans whose hidden boundaries surface only when features interact.

This sequence begins with variables, values, references, and object identity because every later region depends on the difference between a copied value and a shared object. It then moves through the operations that transform those values before introducing larger abstractions. Generics arrive only after declared and run-time types can be separated. Streams arrive only after mutation and collection contracts are visible. Concurrency arrives only after object state, initialization, failures, and execution phases have concrete meanings. The sequence makes each later guarantee state exactly what it adds and what it still cannot add.

The book is trying to convey a method of technical judgment as much as a body of Java facts: predict the event, identify the responsible rule or component, inspect decisive evidence, locate the first divergence, and defend the conclusion's boundary. That method prevents a compiler success from being mistaken for input validation, a

test from being mistaken for a universal proof, an immutable view from being mistaken for unique ownership, or synchronization in one method from being mistaken for a whole-operation invariant.

Dense canonical prose serves readers who already know how ordinary code looks. It should not force every reader through the same remedial detour. When a particular passage remains opaque after an honest prediction and reconstruction attempt, the reader can record the passage, expected result, observed result, and remaining confusion. A personal appendix may then decompress exactly that missing dependency. Repeated reports are evidence for revising the shared chapter; one local difficulty is not a reason to turn the entire book into a beginner sequence.

The final standard is independent use. A reader who merely recognizes each term has not yet mastered the material. A ready reader can analyze an unfamiliar case without the page in view, produce an implementation that preserves its contracts, diagnose the first wrong assumption when evidence disagrees, and say precisely what the result does not establish.

Chapter 1

Variables, Values, References, and Object Identity

Chapter type: Acquisition
Prerequisites: Ordinary declarations, assignments, method calls, simple classes, and arrays
Capabilities introduced: Predict value copying, object aliasing, mutation, reassignment, parameter effects, `final` restrictions, reference identity, logical equality, and shallow array copying
Earlier chapters integrated: None
Later chapters enabled: 2–5, 8–10, 15, and every chapter that reasons about state
Estimated study time: 4–6 hours across four sessions
Runnable sources: `examples/ch01/src/`, `examples/ch01/test/`, and `examples/ch01/lab/`

Compressed thesis

A Java variable is a typed storage location. Its current value is either a primitive value or a reference value. A reference value may refer to an object—a class instance or an array—or it may be the null reference, which refers to no object. Assignment copies a value into the variable, field, or array component selected by the left-hand side. Copying a reference value does not copy the object; it can create another access path to the same object. A state change made through one such alias is observable through the others.

A method invocation also passes values. Each parameter is a new variable for that invocation, initialized with the corresponding argument value. Reassigning a reference parameter therefore changes only the parameter variable, while writing a field or array component through the parameter can mutate the caller's object. A `final` reference prevents later assignment to that particular variable; it does not make the referenced object immutable or remove other aliases.

Reference `==` asks whether two values are the same reference or are both null. An `equals` method asks whatever logical-equivalence question its class contract defines. Array cloning creates a new outer array and copies its component values, so a cloned array of references can still share all of its element objects. These rules describe Java-language behavior and an operational object graph; they do not claim a particular physical heap layout or an ownership system that Java does not provide.

1. Typed storage locations hold values

The Java Language Specification defines a variable as a storage location with an associated type. Local variables, parameters, instance fields, static fields, and array components are all variables, but they are created and initialized under different rules. A local variable must be definitely assigned before use. Fields and array components receive default values when their containing object or array is initialized. Later chapters will trace those timing rules; this chapter begins with the common fact that an assignment stores a value in a selected location.

Java divides values into two families. A variable of primitive type holds a value of that exact primitive type. The primitive types are `boolean`, `byte`, `short`, `char`, `int`, `long`, `float`, and `double`. A variable of reference type holds either a reference to a compatible object or the null reference. Class types, interface types, type variables, and array types are reference types.

Do not collapse a variable, its value, and a referenced object into one entity. In the declaration `int count = 7`, `count` is a variable and `7` is the primitive value stored there. In `Counter counter = new Counter(1)`, `counter` is a variable, the value stored in it is a reference, and the `Counter` created by `new` is an object with its own identity and field state.

Predict before running: After `second` is incremented, what values do the two variables hold? Which location did `++` update?

```
int first = 7;
int second = first;
second++;

System.out.printf("primitive: first=%d second=%d%n", first, second);
```

Output:

```
primitive: first=7 second=8
```

The initializer for `second` copies the current `int` value from `first`. The postfix increment writes a new `int` value into `second`; it does not mutate the number seven and cannot redirect the already completed initialization of `first`. Primitive assignment therefore cannot create shared mutable object state. Two primitive variables may contain equal values, but neither is an access path to the other.

The declared type matters before execution. It limits which values a variable may hold and which operations an expression may request. Chapter 2 will derive conversions in detail. For now, keep the phase boundary visible: a successful assignment means the compiler accepted the source relationship, subject to special boundaries such as unchecked operations and run-time array checks. It does not mean an incoming value satisfies an application rule such as “port is between 1 and 65535.”

2. Reference values preserve access to objects

An object is either a class instance or an array. Each successful object-creation operation produces an object with identity. Object state consists of its instance fields or, for an array, its component variables. A reference value permits operations on the object to which it refers. The language permits many reference values to refer to the same object.

Two access paths are **aliases** when their current reference values refer to the same object. Aliasing is a relationship among paths and one object, not a claim that the variables have merged. Each variable remains a separate storage location and can later receive a different reference value if assignment is permitted.

Predict before running: Does incrementing through `alias` change the value later read through `primary`? Are `primary` and `alias` the same variable?

```
Counter primary = new Counter(1);
Counter alias = primary;
alias.value++;

System.out.printf(
    "alias: same=%b primary.value=%d alias.value=%d%n",
    primary == alias,
    primary.value,
    alias.value
);
```

Output:

```
alias: same=true primary.value=2 alias.value=2
```

The assignment to `alias` copied the reference value held by `primary`. Only one `Counter` expression executed, so only one `Counter` object was created. The field update selected the `value` field inside that object. A later read through either reference observes the field's new value. `primary == alias` is true because both operands produce references to the same object, not because the two local variables are one location.

An object graph is a useful operational representation. Draw variables and object fields as labeled locations. Draw an object as an identity-bearing node. A primitive value may be written directly in a location; a non-null reference value can be drawn as an edge to a node. In the preceding case, `primary` and `alias` have separate edges to the same `Counter` node, whose `value` field contains `2`. The graph predicts assignment, identity, and mutation. It does not assert whether a virtual machine uses a raw address, a handle, compressed object pointers, escape analysis, or another physical representation.

The null reference is a reference value that reaches no object. It can be copied and compared. An operation that needs an object, such as instance field access or instance method invocation, cannot follow null to a node and generally completes abruptly with `NullPointerException`. Null is not an object with an empty state, and two null-valued variables do not alias an object merely because `a == b` is true. Chapter 11 handles the failure boundary; Chapter 19 handles API null policy.

3. The assignment target determines what changes

The slogan "assignment changes the variable" is too broad because fields and array components are also variables. The left-hand operand identifies the location to update. If `primary` is a local variable, `primary = replacement` stores a different reference value in that local variable. If `primary.value` is an instance field, `primary.value = 9` follows `primary` to an object and stores an `int` in that object's field. If `items[0]` is an array component, `items[0] = replacement` changes an element location inside the array object.

This book uses **reassignment** for storing a new value in an existing location and **rebind** informally when a reference variable changes which object it reaches. Mutation means a state change to an existing object. Rebinding a local reference changes one edge in the object graph. Mutating a field changes state inside a node. Assigning a reference into an object field is a mutation of the containing object and also redirects one edge from that object. Naming the exact location avoids treating every `=` as the same state transition.

Predict before running: After the second statement, which object does `left` reach? Which object does `right` reach? What happened to the original object's field?

```
Counter left = new Counter(3);
Counter right = left;

left = new Counter(40);
right.value = 4;

System.out.printf(
    "reassign: same=%b left.value=%d right.value=%d%n",
    left == right,
    left.value,
    right.value
);
```

Output:

```
reassign: same=false left.value=40 right.value=4
```

The second `new` expression creates a second object, and assignment redirects only `left`. `right` continues to refer to the first object. The field assignment through `right` changes that first object's state. Nothing in the reassignment to `left` deletes or resets the first object; whether an object later becomes eligible for garbage collection depends on reachability through all live paths, not on one variable name. Collector algorithms and liveness diagnostics are outside this chapter.

Aliasing is not itself a defect. Two collaborators may intentionally share a connection, cache entry, mutable buffer, or domain entity whose stable identity matters. Aliasing enlarges the reasoning boundary: to prove that shared state stays unchanged, a programmer must account for every path that can perform a relevant mutation during the relevant lifetime. An API therefore needs a mutation and ownership policy even though Java's ordinary reference types do not encode unique ownership.

4. Parameters receive argument values

For each method invocation, Java creates a new parameter variable for every declared parameter. Each parameter is initialized with the corresponding argument value. That rule applies uniformly: an `int` argument supplies an `int` value; an object-producing argument supplies a reference value; a null-valued argument supplies null. Java does not give the method an assignable alias for the caller's local variable.

Predict before running: The method mutates through its parameter and then reassigns the parameter. What state remains in `caller`? What state is in the returned object? Are they identical?

```
static Counter mutateThenReplace(Counter parameter) {
    parameter.value += 10;
    parameter = new Counter(99);
    parameter.value++;
    return parameter;
}

Counter caller = new Counter(2);
Counter returned = mutateThenReplace(caller);

System.out.printf(
    "call: caller=%d returned=%d same=%b%n",
    caller.value,
    returned.value,
    caller == returned
);
```

Output:

```
call: caller=12 returned=100 same=false
```

Argument evaluation copies the reference to the first `Counter` into the new parameter variable. The first field assignment follows that reference and mutates the shared object from `2` to `12`. The next assignment stores a reference to a newly created object in the parameter variable only. The caller's local variable is a different location, so it still reaches the first object. The final increment changes the second object, and the return statement copies its reference value into the calling expression; initialization then stores that value in `returned`.

"Objects are passed by reference" predicts the first mutation but commonly predicts that `parameter = new Counter(99)` should redirect `caller`. That prediction is false. "Object references are passed by value" predicts both operations, provided the sentence is unpacked: the passed value is a reference, copying it creates an alias, field access can mutate the shared object, and assignment to the parameter changes only the call-local parameter variable.

A method can request that a caller adopt a replacement object by returning its reference, but the caller performs the assignment: `caller = mutateThenReplace(caller)`. The method cannot directly reassign that caller local through an ordinary Java parameter. A method can still modify caller-visible locations reached through references—for example, an element of a supplied array or a field of a supplied holder object. That is mutation of a shared object, not pass-by-reference parameter semantics.

5. `final` restricts a location, not a reachable graph

A `final` variable may be assigned only under the language's final-variable rules. Once a final local variable has been assigned, it cannot be the target of a later assignment or increment. For a reference variable, this protects one edge in the object graph: the variable cannot later store null or a reference to a different object. It does not freeze the object's fields, recursively freeze referenced objects, prevent methods from mutating the object, or prevent another alias from doing so.

Predict before running: Does this compile? If it does, what value is printed? Which statement would fail compilation if uncommented?

```
final Counter stableReference = new Counter(5);
Counter writableAlias = stableReference;

writableAlias.value += 2;
stableReference.value++;
// stableReference = new Counter(0);

System.out.printf("final: value=%d%n", stableReference.value);
```

Output:

```
final: value=8
```

Both field assignments are legal because neither assigns the local variable `stableReference`; they update the field of the shared `Counter`. The commented assignment is illegal because it attempts to store a new reference value in a final local variable. The compiler enforces that restriction at the assignment target. The object has no general knowledge that one path reaching it was stored in a final variable.

Java uses `final` for several distinct declarations: local variables, parameters, fields, methods, and classes. Their restrictions are related by the keyword but not interchangeable. A final method cannot be overridden, and a final class cannot be subclassed; neither claim makes every instance deeply immutable. A final field also participates in special construction and memory-model rules, but Chapter 5 and Chapter 15 must establish the conditions before this book relies on those guarantees.

An immutable design requires a contract about observable state, not merely a final reference. Such a design may combine final fields, construction invariants, defensive copies, immutable component types, no mutating methods, and safe publication. Its depth is explicit: an immutable outer object can still expose a mutable array and thereby allow observable change. Chapter 19 turns those observations into an API policy.

6. Identity and logical equality ask different questions

For reference operands, `==` returns true when both values are null or both refer to the same object. It does not inspect fields. Logical equivalence is expressed by an `equals` method whose class-specific contract decides which state matters. The default `Object.equals` behavior is identity equality, but classes may override it. `String`, for example, defines equality in terms of its character sequence.

Predict before running: Which comparisons are true? Why can the two `String` values be logically equal without being identical?

```
String textA = "Java";
String textB = new String("Java");

Counter counterA = new Counter(1);
Counter counterB = new Counter(1);
Counter counterAlias = counterA;

System.out.printf(
    "equality: textIdentity=%b textEquals=%b counterIdentity=%b aliasIdentity=%b%n",
    textA == textB,
    textA.equals(textB),
    counterA == counterB,
    counterA == counterAlias
);
```

Output:

```
equality: textIdentity=false textEquals=true counterIdentity=false aliasIdentity=true
```

The string constructor creates a distinct `String` object, so the reference identity comparison is false. Both strings represent the same character sequence under `String.equals`, so logical equality is true. The two `Counter` constructions also produce distinct objects. `Counter` does not override `Object.equals` in the runnable example, so even `counterA.equals(counterB)` would remain false despite equal-looking fields. The alias comparison is true because it compares two references to the one object created for `counterA`.

String literals introduce interning rules, but code should not use `==` as content comparison merely because two literal references may be identical. That observation depends on how the strings were obtained and answers the identity question. The desired application question is normally sequence equality, which `String.equals` defines directly.

Full equality design requires more structure than this chapter has introduced. An overridden `equals` method must obey its contract, equal objects must produce equal hash codes, and ordering may need to agree with equality. Mutating state used by equality can break collection lookups. Chapter 9 acquires those relationships. The durable rule here is smaller: decide which question the program needs, then use the operation whose contract asks that question.

7. Array copies can preserve nested aliases

An array is an object, and every array component is a variable. An array of primitive type stores primitive values in its components. An array of reference type stores reference values. A two-dimensional array type such as `int[][]` is therefore an array whose components contain references to `int[]` objects; Java does not require one rectangular block containing every integer.

Calling `clone()` on an array creates a new array object of the same run-time type and copies the component values. For a primitive array, those copied components are primitive values. For an array of references, the copied components are reference values, so the old and new outer arrays initially refer to the same element objects. This is a shallow copy.

Predict before running: Which arrays have new identity? Which row remains shared? Which assignment affects the original grid, and which affects only the copied outer array?

```
int[][] grid = {
    {1, 2},
    {3, 4}
};

int[][] outerCopy = grid.clone();
outerCopy[0][0] = 9;
outerCopy[1] = new int[] {7, 8};

System.out.printf(
    "arrays: outerSame=%b firstRowSame=%b original00=%d original10=%d copy10=%d%n",
    grid == outerCopy,
    grid[0] == outerCopy[0],
    grid[0][0],
    grid[1][0],
    outerCopy[1][0]
);
```

Output:

```
arrays: outerSame=false firstRowSame=true original00=9 original10=3 copy10=7
```

The clone operation creates a new outer array, so `grid == outerCopy` is false. Copying component zero copies a reference to the first row, so both outer arrays initially reach the same `int[]`; changing its component zero is visible through `grid`. Assigning `outerCopy[1]` changes a component of the copied outer array, redirecting only that component to a new row. The original outer array still refers to its original second row.

The phrase "copy the array" is incomplete until it names which array objects must be new. A fully independent copy of this particular graph requires a new outer array and a new row array for every row that must not be shared. General object graphs can contain cycles, repeated aliases, objects with identity-sensitive contracts, and resources that cannot sensibly be duplicated. Java supplies no universal deep-copy operator because "deep" is a domain policy, not merely a recursion depth.

Selective copying can be useful. If a transformation changes only one row and the program guarantees that unchanged rows will not be mutated, a new outer array plus one new row may preserve safe sharing and avoid unnecessary work. If callers can mutate the shared rows, the same topology violates snapshot independence. The operation is identical; the surrounding mutation contract determines whether the sharing is valid.

False models, repaired

- **"A reference variable contains the object."** It contains a reference value; the object has a separate identity and may be reached through other values.
- **"Assigning an object variable copies the object."** Assignment copies the current reference value and may create another alias.
- **"Java passes objects by reference."** Invocation initializes a new parameter variable with the argument value; a reference value can preserve access to a shared object, but parameter reassignment stays local.
- **"Every assignment is a rebind."** The left-hand side may select a local, a field, or an array component. A field or component assignment mutates its containing object.
- **"`final` makes an object immutable."** A final reference variable cannot later hold another value; reachable object state may still change.
- **"`==` compares object contents."** Reference `==` compares identity or two null references. Logical equality comes from the class's `equals` contract.
- **"Cloning an array recursively clones its elements."** Array cloning creates one new array and copies component values; copied references retain element aliases.
- **"Null is an empty object."** The null reference refers to no object, so an object-requiring operation cannot follow it to state or behavior.

Mastery questions

Commit to a result and causal explanation before compiling any constructed case.

1. `long b = a; b++;` follows `long a = 4;`. Predict both values and identify every updated location.
2. Two local variables refer to one mutable `Box`. One variable receives a new `Box`; the other changes a field. Draw the graph after each statement.
3. A method sets `parameter.enabled = true`, assigns `parameter = null`, and returns. What can the caller observe, and why does no caller variable become null?
4. Give one concrete observation that the phrase "objects are passed by reference" predicts incorrectly.
5. A final local refers to an array. Which of these are permitted: replacing the local's value, replacing element zero, and changing a field of the object stored at element zero? Name the assignment target in each case.
6. Two separately constructed `Customer` objects contain the same database ID. What does `==` establish? What additional contract would be required for logical equality?
7. Why does `new String("x")` make an identity comparison a different question from `String.equals`?
8. Draw a graph for `Widget[][] copy = original.clone()`. Which nodes are necessarily new, and which may be shared?
9. Diagnose a function advertised as returning an independent snapshot when it clones only an outer array of mutable objects.
10. Design a selective copy for a three-level configuration graph where only one leaf changes. Which path objects must be new if old and new roots must be independently stable?
11. A variable has interface type `Runnable` and refers to an instance of a named class. Separate the variable's compile-time type, its current reference value, the object's identity, and its run-time class.
12. Explain why a final reference can reduce accidental reassignment without reducing the number of aliases that can mutate the object.
13. A null-valued variable compares equal with another null-valued variable. Why is "the two variables alias the same null object" an invalid explanation?
14. Describe one API where shared mutation is intentional. State which aliases may mutate, during what lifetime, and which invariant contains the risk.

The closed-book workspace, transfer cases, mismatch-report format, and retrieval schedule are in `exercises/ch01.md`.

Implementation lab — Copy-on-Write Profile Update

Implement `StudentProfileUpdater.updateTheme` in `examples/ch01/lab/`. The input graph contains a `Profile`, its mutable `Preferences`, and an audit-code array. Return a new outer `Profile` with the requested theme without mutating the input. The result must contain a new `Preferences` object, because its state differs. Preserve the `String` ID reference and the audit-array reference, because the contract designates both as unchanged shared values. This is selective copying, not a claim of deep immutability: callers capable of mutating the shared audit array can still affect both profiles.

Before coding, commit to all four identity relationships: input/result profiles, input/result preferences, input/result IDs, and input/result audit arrays. The evaluator also checks values and proves the input preferences were not mutated. After your implementation passes, run the evaluator against `FaultyProfileUpdater`, which creates a new outer profile but mutates the still-shared preferences object. Explain the first operation at which that implementation violates the contract.

Presentation prompt

Give a 10–15 minute explanation from sparse notes. Reconstruct the relationship among a variable, its type, its current value, a reference, and an object. Include one committed prediction involving method-parameter reassignment, a graph derivation for the two-dimensional array clone, the false model that `final` means immutable, the boundary between `==` and `equals`, one legitimate use of shared mutation, and one fact the Chapter 1 tests cannot prove universally. End by naming the mechanism that still feels least reliable. The recording checklist is in `presentations/prompts/ch01.md`.

Ready-when test

Proceed when, without the chapter open, you can predict every canonical output; draw the object graph after reassignment, mutation, a method call, and an array clone; explain pass-by-value without a special exception for objects; separate a final variable from object immutability; choose deliberately between identity and logical equality; implement and pass the copy-on-write lab; diagnose the faulty updater at its first divergent write; and state what the compiler and tests establish and what they do not. A vocabulary-perfect answer that predicts the wrong object identity or mutation is not ready.

Personal appendix index

No personal appendices have been generated. This is expected at the gold-chapter milestone.

Glossary

This glossary records the canonical sense of load-bearing terms. Chapters still define each term at first consequential use.

- **alias** — one of two or more access paths whose current reference values refer to the same object.
- **argument** — an expression supplied at an invocation; its evaluated value initializes a new parameter variable.
- **assignment** — an operation that stores a compatible value in a variable, field, or array component identified by the left-hand operand.
- **compile-time type** — the type assigned by the language rules to a variable or expression during compilation; it constrains accepted operations and possible values.
- **declared type** — the type written or inferred for a declaration, considered as a compile-time type.
- **identity** — the property by which an object remains that particular object despite state changes and differs from another object with equal-looking state.
- **immutability** — a contract that specified state cannot change after a defined point; the contract must name its depth, enforcement mechanism, and observable boundary.
- **mutation** — a state change to an existing object, including a write to one of its instance fields or array components.
- **object** — a class instance or an array at run time.
- **parameter** — a call-local variable created for an invocation and initialized with the corresponding argument value.
- **primitive value** — a non-reference value of one of Java's primitive types.
- **reference type** — a class, interface, type-variable, or array type whose values are references or the null reference.
- **reference value** — a value that refers to an object, or the null reference, which refers to no object.
- **reassignment** — an assignment that changes the value stored in an already existing variable, field, or array component.
- **run-time class** — the class of the object referred to by a non-null reference value during execution.
- **shallow copy** — a new outer object whose component values are copied without recursively copying every referenced object.
- **state** — the values stored in an object's instance fields or array components at a given point in execution.
- **variable** — a typed storage location. Local variables, parameters, fields, and array components are distinct kinds with different creation and initialization rules.